

Sovereign On-Premise Agentic AI Workbench

SIH 2026 • Problem Statement 26117

Final Research Synthesis

Consolidated engineering, architecture, security, model, feasibility and demo strategy from nine independent AI research reports.

Research Corpus: KIMI K3 • Qwen 3.8 Max • Gemini 3.1 Pro Extended Thinking • Claude Sonnet 5 • Claude Haiku 4.5 Extended • GPT-5.6 Sol • GPT-5.6 Sol Deep Research • Grok 4.5 • Perplexity

Research Date: 22 August 2026

Executive Verdict & Decision Summary

Core Principle

Build a workflow-oriented, approval-gated, air-gapped agent runtime, not a generic local-chat wrapper.

The strongest SIH implementation is a single-node workbench with: - Capability registry - Deterministic model router - Local OCR/VLM pipeline - Hybrid RAG system - Hardened code sandbox - Real artifact generation - Live sovereignty proof plane

Research Alignment

All nine reports strongly agree on this architecture. Main disagreements are implementation choices: - Ollama vs vLLM - Qdrant vs LanceDB - Which current open-weight model is best

The synthesis resolves these using SIH reliability, judge-visible depth and build feasibility as criteria.

Final SIH Stack

Inference: Ollama first, vLLM-compatible abstraction

Models: Qwen-family general/coder models sized to GPU; Qwen-VL-class 7B/8B for vision

OCR: PaddleOCR

Embeddings: BGE-M3; bge-reranker-v2-m3

Vector DB: Qdrant

Orchestration: LangGraph-style state machine; FastAPI gateway

Sandbox: Rootless Docker/Podman + seccomp

Artifacts: python-docx/openpyxl/python-pptx

Sovereignty: nftables/firewall + tcpdump/eBPF

Strategic Scope Decision

Three workflows only: - Inspection report → Approval note (multimodal + RAG + artifact) - Coding → Sandbox → Tests → Repair (routing + tools + sandbox verification) - Sovereignty proof (live egress blocking demonstration)

The research repeatedly recommends three polished workflows instead of a feature graveyard. This focus transforms generic features into defensible product.

Requirements Decomposition

All nine research reports independently converge on this critical requirement set:

CRITICAL: Fully local/air-gapped with egress enforcement and live telemetry

CRITICAL: Multi-model with automatic selection and routing receipt

CRITICAL: Agentic execution — Plan → tools → verification → correction → artifact

CRITICAL: Multimodal processing of PDF/image/P&ID/handwriting locally

CRITICAL: Local knowledge base with SOP/manual retrieval and citations

CRITICAL: Real deliverables in DOCX/XLSX/PPTX/code/calculation report formats

CRITICAL: Sandboxed code execution with isolation and resource limits

STRONG: Auditability — complete trace of model, tools, sources, tests, artifact

OPTIONAL: RBAC/SSO/multi-user (architecture-ready, explicitly outside MVP)

Strategic Consequence

Completeness matters more than model size. A smaller model executing every required stage is more defensible than a giant model doing only chat.

Research Contributions Overview

Each report contributed distinct, high-value insights:

KIMI K3: Detailed threat model; capability registry + hybrid router; deterministic state machine

Qwen 3.8 Max: Three-workflow blueprint; 16–24 GB target; visible sovereignty layer

Gemini 3.1 Pro: vLLM architecture; eBPF/Tetragon monitoring; hard iteration cap

Claude Sonnet 5: 36-hour roadmap; demo pacing; rehearsal discipline

Claude Haiku 4.5: Coding/debugging loop; multimodal document-to-action workflow

GPT-5.6 Sol: Evidence-classification; approval-gated workflow; local RAG stack

GPT-5.6 Deep: 2026 model landscape; hardware-conscious selection; air-gap emphasis

Grok 4.5: Modern positioning; Qwen3/VLM fusion; auditable air-gap

Perplexity: Competitive landscape; performance targets; bottleneck analysis

This was treated as synthesis, not voting. Where reports conflicted, the final choice favors reliability during SIH while preserving a clean upgrade path.

Final System Architecture

WORKBENCH: React/Next.js or SvelteKit UI with task input, file upload, live trace, routing receipt, artifact preview and sovereignty dashboard.

ORCHESTRATION: FastAPI → classifier → capability registry → router → bounded state-machine agent → typed tool registry → approval gates.

KNOWLEDGE BASE: Docling/PyMuPDF → OCR/layout → VLM fallback → chunking → BM25 + dense retrieval → reranking → citation-grounded context.

INFERENCE: Ollama for SIH simplicity, exposed behind a provider interface so vLLM can replace it.

TOOLS: Workspace filesystem, calculations, spreadsheets, document generation, code execution and validation.

SOVEREIGNTY: Host egress denial + isolated service networking + sandbox network disabled + telemetry + hashes + vendored dependencies.

The strongest convergent blueprint uses exactly this combination: capability registry/router, deterministic orchestration, local OCR/VLM, hybrid RAG, sandboxed execution, artifact generation and visible network isolation.

Hardware & Model Strategy

Hardware Profile Selection

8–12 GB VRAM: 7–8B Q4 text, 3–4B VLM — Fallback; basic RAG; no high-end multimodal

16 GB VRAM: 14B Q4/Q5 text, 7B VLM — Lower-bound competitive target

24 GB VRAM: 27–32B Q4 text, 7–8B VLM — BEST SIH TARGET (RTX 3090/4090 class)

48 GB VRAM: 32B Q6+ text, 12–34B VLM — High-quality demo tier

Key Insights

24 GB RTX 3090/4090 identified as ideal for hackathon (~20 GB for 32B Q4 + ~5 GB for 7B VLM).

A 24 GB card can keep a 7B VLM resident while hot-swapping the text model.

Do not publish exact throughput/VRAM claims without re-benchmarking.

Recommended Model Portfolio

General Reasoning: Qwen 14B/27B/32B (primary), 7–14B general (fallback) — Planning, reasoning

Coding: Qwen coder/Devstral (primary), 7–14B coder (fallback) — Code generation, tests, debug

Vision: Qwen-VL 7B/8B (primary), Gemma/Pixtral (fallback) — Scans, photos, diagrams

OCR: PaddleOCR (primary), Tesseract (fallback) — Printed text, tables, layout

Embeddings: BGE-M3 (primary), nomic-embed (fallback) — Dense retrieval

Reranker: bge-reranker-v2-m3 (primary), cross-encoder (fallback) — Evidence refinement

Decision: Benchmark candidates before freezing the registry. The platform architecture must make model replacement boring.

Agent Architecture

Use a bounded deterministic state machine, not an unrestricted agent swarm.

State Machine Flow: INTAKE → CLASSIFY → PLAN → RETRIEVE → TOOL → VERIFY → REPAIR → ARTIFACT → APPROVAL → COMPLETE

State Machine Guarantees

- Every transition has typed inputs/outputs
- Timeout on every state
- Maximum step count enforced
- Explicit termination conditions
- Tools are allowlisted
- Retrieved documents are data, never instructions
- Destructive writes require approval

Research Consensus

KIMI: Deterministic LangGraph-style state machine is more reliable, observable and demoable than free-running ReAct loops.

Gemini: State manager + tool registry + hard iteration cap with local tool sandbox and explicit security monitoring.

36-Hour SIH Build Plan

Pre-Hackathon Preparation (Critical)

- Pre-stage models
- Prepare synthetic corpus
- Build templates
- Create sandbox image
- Implement router interface
- Test firewall behavior

Timeline

0–4h: Docker, firewall, models, dependency freeze → No UI polish

4–8h: FastAPI, adapter, registry, router → Routing receipt works

8–14h: State machine, tools, workspace → One full tool chain works

14–20h: RAG + OCR + VLM + corpus → Citations work

20–25h: DOCX workflow → Workflow 1 runnable

25–29h: Sandbox + tests + repair → Workflow 2 runnable

29–32h: Sovereignty dashboard + proof → Proof is live

32–34h: Fallbacks, caching, validation → No new features

34–36h: Full rehearsal + presentation → FEATURE FREEZE

Critical Milestone

After hour 20, stop working as isolated specialists and repeatedly run complete workflows. Integration is the product.

Security Threat Model

Prompt injection: System/tool instructions separated; allowlisted tools; approval gates

Malicious PDF/exploit: Isolated parsing; page/memory limits; patched parsers

KB poisoning: Source trust metadata; scoped retrieval; citation transparency

Tool abuse: Workspace-jail; read-only KB; typed arguments

Sandbox escape: Rootless, dropped caps, seccomp, no socket; microVM upgrade

Supply chain: Vetted sources, checksums, safe formats, offline manifest

Dependency telemetry: Vendored dependencies + egress denial

Infinite loops: Step budget, timeout, termination state, human escalation

Sovereignty Proof Plane

Core Principle

“Everything is local” is a claim. Technically unavailable network access is evidence.

Implementation

Host Level: Default-deny outbound firewall

Service Level: Only required local interfaces exposed

Sandbox Level: No network access

Telemetry: tcpdump or eBPF/Tetragon live logging

Supply Chain: Pre-staged models/dependencies, checksums and offline manifests

Best Demo Strategy

Deliberately attempt an outbound request from the sandbox. Show the request blocked, the packet event recorded and the normal workflow continuing with zero external traffic.

Three Final Demo Workflows

Workflow 1: Inspection → Approval Note

End-to-End: PDF → OCR → VLM → RAG → calculation → draft → DOCX

Test Case: Synthetic inspection report with OCR challenges, one measured value, one finding and one relevant SOP. Agent must cite SOP, perform deterministic calculation, draft approval note and produce DOCX.

Workflow 2: Coding → Verified Output

End-to-End: Request → route → generate → sandbox test → repair → package

Test Case: “Add pagination to this endpoint.” Retrieve file, generate change, run tests, encounter seeded failure, repair it and return passing result.

Workflow 3: Sovereignty Proof

End-to-End: Outbound attempt → blocked → event logged; normal workflow → zero external traffic

What NOT to Build

Explicit Scope Exclusions: - Cloud dependency - Giant-model vanity deployment - Multi-agent swarms - Custom inference engine or vector DB - Full P&ID connectivity parser or CAD parser - Fine-tuning during SIH - Production SSO/RBAC or Kubernetes/HA - Browser automation - Sovereignty dashboard without real egress blocking

Also avoid exposing hidden chain-of-thought as product feature—show concise execution steps, tool calls, evidence and outcomes instead.

Final Verdict & Recommendations

Problem Characterization

SIH26117 is a high-difficulty, high-ceiling problem with unusually strong alignment between technical depth and demo impact. It is resource-heavy enough to discourage shallow teams, but mature open-source infrastructure makes a credible MVP feasible.

Winning Strategy

Do not implement every noun in the problem statement. Build one narrow vertical slice that crosses every critical requirement end-to-end.

Best Realistic Product

A sovereign local workbench that takes confidential industrial document or coding request, automatically selects appropriate open-weight model, performs bounded multi-step workflow with local tools/knowledge, verifies result, emits real artifact, and continuously demonstrates zero external network access.

Critical Differentiators (Priority Order)

Technical: Capability registry + deterministic router + bounded agent state machine

Demo: Scanned inspection report → grounded approval note → DOCX

Security: Enforce egress denial and demonstrate it live

Scope: No swarms, no custom DBs, no training, no enterprise IAM, no full CAD/P&ID

Market: Compete on complete sovereign execution environment, not “local ChatGPT”

Pre-Build Validation Checklist

- Model files pre-staged
- Dependency bundle frozen
- Synthetic industrial corpus prepared
- DOCX template validated
- Sandbox image built
- Routing receipt implemented
- Packet monitor tested
- Offline clean-machine rehearsal completed

Final Note

Validation before implementation—re-check exact model cards, licenses, quantized memory footprints, tool-calling behavior, OCR quality, sandbox behavior and firewall rules on exact hardware available for SIH. The model landscape changes too quickly to treat a research report as frozen hardware specification.